Custom Dart Actions in FlutterFlow

Author: Wahab Waris

When Visual Building Isn't Enough: Writing High-Performance Custom Dart Actions in FlutterFlow

Written by Wahab Waris

Introduction

FlutterFlow's drag-and-drop builder, action flow editor, and built-in widgets cover a huge percentage of app-building needs. But every developer eventually hits a wall: a complex calculation, a third-party SDK that has no native FlutterFlow integration, a performance-critical loop, or a piece of business logic that's just too messy to express with visual action blocks.

This is where Custom Dart Actions come in. They let you drop straight into raw Dart code, write exactly the logic you need, and plug the result right back into your visual app — getting the best of both worlds: speed of visual development plus the full power of Dart/Flutter.

In this tutorial, you'll learn:

1. What Custom Dart Actions are and when to use them
2. How to create your first Custom Action from scratch, step-by-step
3. How to pass arguments in and return values out
4. How to use external packages (pub.dev) inside a Custom Action
5. A real working example: a radius/distance filter function
6. Common errors and how to fix them
7. Best practices for writing clean, reusable, high-performance actions

By the end, you'll be confident writing your own Custom Actions for any project.

1. What Are Custom Dart Actions?

A Custom Action in FlutterFlow is a standalone Dart function that you write yourself. It lives outside the visual Action Flow editor but can be called from anywhere inside it — buttons, page load triggers, backend calls, widget states, you name it.

Use Custom Actions when you need to:

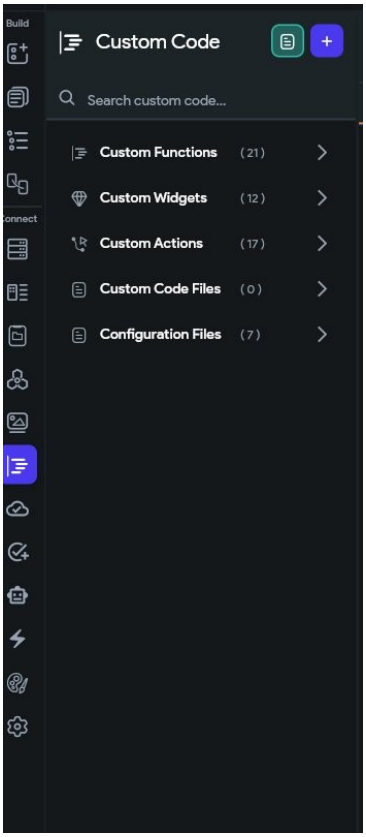
- Perform complex math/logic (distance calculations, sorting, filtering)
- Call a third-party API or SDK not natively supported
- Manipulate data structures (JSON parsing, list transformations)
- Run platform-specific code
- Improve performance by avoiding multiple chained visual actions

Tip: If you find yourself chaining 8–10 visual actions just to transform some data, that's usually a sign a Custom Action would be cleaner and faster.

2. Creating Your First Custom Action — Step by Step

Step 1: Open the Custom Code Section

8. In the left-hand navigation panel of your FlutterFlow project, scroll down to Custom Code.
9. Click on Custom Actions.



The Custom Code panel — Custom Actions, Custom Widgets, Custom Functions and related sections.

Step 2: Create a New Action

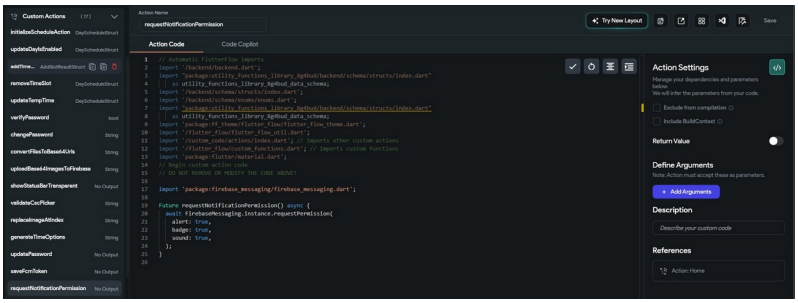
10. Click the + Add button (usually top-right of the Custom Actions panel).
11. Give your action a clear, descriptive name — e.g., calculateDistance.

- Naming convention: use camelCase, and make the name describe what it returns or does.

Step 3: Understand the Code Editor Layout

When the editor opens, you'll see:

- Action Name (top)
- Return Type dropdown (String, Integer, Double, Boolean, List, DataType, etc.)
- Arguments section (where you define inputs)
- Code Editor (the main Dart code area)

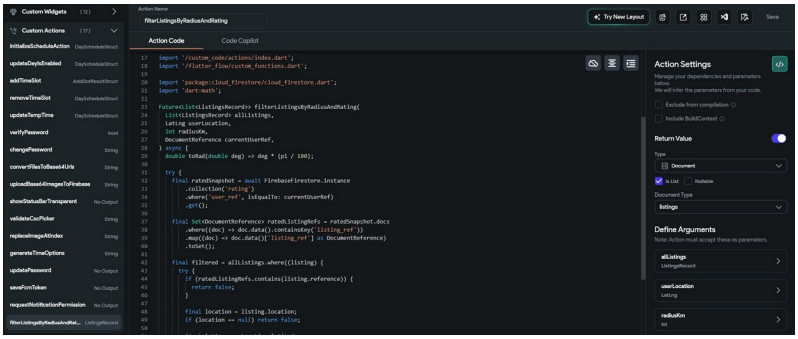


requestNotificationPermission action — code editor, Action Settings, and arguments panel.

Step 4: Define Arguments (Inputs)

Arguments are the values your action receives from the app — for example, a user's latitude/longitude, a list of items, or a search string.

12. Click + Add Argument.
13. Set the Argument Name (e.g., userLat).
14. Set the Type (e.g., double).
15. Repeat for each input your function needs.



filterListingsByRadiusAndRating action showing defined arguments such as allListings, userLocation, and radiusKm.

Step 5: Set the Return Type

Decide what your function will give back to your app:

- double — for calculated numbers (e.g., distance)
- bool — for true/false checks
- List — for filtered/sorted lists
- String, int, DateTime, etc.

Select this from the Return Type dropdown at the top of the editor.

Step 6: Write Your Dart Code

Here's a simple, real-world example — calculating the distance (in kilometers) between two GPS coordinates using the Haversine formula:

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/backend/schema/util/firestore_util.dart';
import '/backend/schema/enums/enums.dart';
import '/flutter_flow/flutter_flow_util.dart';
import 'dart:math';
```

```
double calculateDistance(
  double userLat,
  double userLng,
  double targetLat,
  double targetLng,
) {
  const double earthRadiusKm = 6371;
  double dLat = _degreesToRadians(targetLat - userLat);
  double dLng = _degreesToRadians(targetLng - userLng);
  double a = sin(dLat / 2) * sin(dLat / 2) +
      cos(_degreesToRadians(userLat)) *
      cos(_degreesToRadians(targetLat)) *
      sin(dLng / 2) *
      sin(dLng / 2);
  double c = 2 * atan2(sqrt(a), sqrt(1 - a));
  return earthRadiusKm * c;
}

double _degreesToRadians(double degrees) {
  return degrees * (pi / 180);
}
```

Important: FlutterFlow auto-generates the function signature based on the Action Name, Arguments, and Return Type you set in Steps 2, 4, and 5. Don't rename the main function manually — let FlutterFlow keep it in sync with your settings.

Step 7: Test the Action

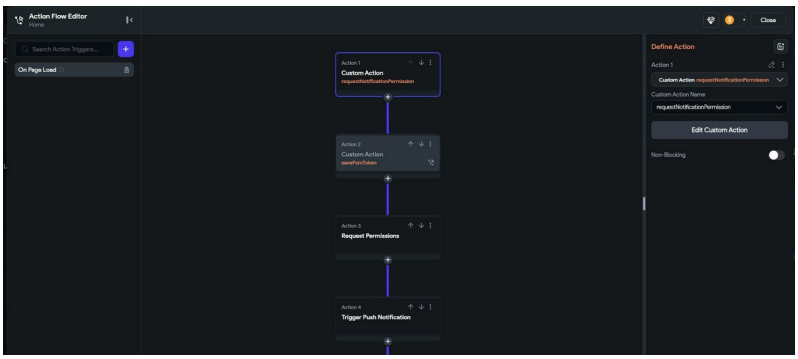
FlutterFlow lets you test Custom Actions directly in the editor before using them anywhere.

16. Click the Test button near the top of the editor.
17. Enter sample values for each argument.
18. Click Run.
19. Check the Result panel for the output — and check the Console for any errors.

If you see a green success result with the expected number, your action is working correctly.

Step 8: Save and Use the Action

20. Click Save (top-right corner).
21. Go to any Action Flow (e.g., a button's onTap action).
22. Click + Add Action → scroll to Custom Actions → select your action by name.
23. Map each argument to a value from your app's state, widget data, or page parameters.
24. Use Action Output Variable to capture the returned value and use it (e.g., display it in a text widget, or use it in a conditional).



Action Flow Editor — a Custom Action (requestNotificationPermission) wired into an On Page Load trigger sequence.

3. Using External Packages (pub.dev) in Custom Actions

Sometimes your logic depends on a package not built into FlutterFlow by default (e.g., geolocator, uuid, crypto).

Step 1: Add the Dependency

25. Go to Settings & Integrations → App Settings → App Dependencies (sometimes called "Custom Packages" or "Pub.dev Dependencies", depending on your FlutterFlow version).
26. Click + Add Dependency.
27. Enter the exact package name and version from pub.dev — e.g., crypto: ^3.0.3.
28. Click Confirm/Add.

Step 2: Import It in Your Custom Action

At the top of your Custom Action code, add:

```
import 'package:crypto/crypto.dart';
import 'dart:convert';
```

Now you can use the package's classes and functions directly inside your action.

If the package isn't recognized, double-check:

- The package name is spelled exactly as on pub.dev
- The version constraint is valid (no typos in ^x.x.x)
- You clicked Save after adding the dependency

4. Real Example: Filtering a List of Listings by Radius

This example combines everything above into something genuinely useful — filtering a list of pet/business listings to only those within a given radius of the user's location.

```
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_util.dart';
import 'dart:math';

Future<List<ListingsRecord>> filterListingsByRadius(
  List<ListingsRecord> allListings,
  double userLat,
  double userLng,
  double radiusKm,
) async {
  List<ListingsRecord> filtered = [];
  for (var listing in allListings) {
    final lat = listing.location.latitude;
    final lng = listing.location.longitude;
    double distance = _calculateDistance(userLat, userLng, lat, lng);
    if (distance <= radiusKm) {
      filtered.add(listing);
    }
  }
  return filtered;
}

double _calculateDistance(
```

```
    double lat1, double lng1, double lat2, double lng2) {
  const double earthRadiusKm = 6371;
  double dLat = (lat2 - lat1) * (pi / 180);
  double dLng = (lng2 - lng1) * (pi / 180);
  double a = sin(dLat / 2) * sin(dLat / 2) +
      cos(lat1 * (pi / 180)) *
      cos(lat2 * (pi / 180)) *
      sin(dLng / 2) *
      sin(dLng / 2);
  double c = 2 * atan2(sqrt(a), sqrt(1 - a));
  return earthRadiusKm * c;
}
```

Setup in FlutterFlow:

- Return Type: List → select your Firestore collection's data type (e.g., Listings)
- Arguments:
  1. allListings → List<DocumentReference>
  2. userLat → double
  3. userLng → double
  4. radiusKm → double

5. Common Errors & How to Fix Them

- **The function 'X' isn't defined:** Helper function name doesn't match what's called — **Fix:** Make sure helper function names match exactly (case-sensitive)
- **Type 'X' not found:** Missing import for a custom Data Type or struct — **Fix:** Add the correct import path from /backend/schema/...
- **Undefined name 'pi' / 'sqrt':** Missing dart:math import — **Fix:** Add import 'dart:math'; at the top
- **Package not recognized:** Dependency not added or not saved — **Fix:** Re-check App Dependencies, re-save, and re-open the editor
- **Returns null unexpectedly:** Return type mismatch — **Fix:** Ensure your function's return statement matches the selected Return Type exactly
- **Action doesn't appear in Action Flow list:** Forgot to save — **Fix:** Click Save in the Custom Action editor first

6. Best Practices for Writing Custom Actions

29. Keep actions small and single-purpose. One action = one job. Easier to test, debug, and reuse.
30. Use descriptive names for both the action and its arguments — your future self (and teammates) will thank you.
31. Always test before connecting to the UI — catch errors early in the isolated test panel.
32. Avoid heavy loops on the main thread for large datasets — consider using Future and async/await for anything that takes time (API calls, large filtering operations).
33. Comment your code, especially helper functions — this matters a lot when other developers (or future you) need to maintain it.
34. Reuse helper functions across actions by keeping consistent naming and structure — you can copy-paste shared helpers like _calculateDistance between actions.

35. Don't hardcode values — always pass dynamic data through arguments so the action stays reusable across different pages/widgets.

Conclusion

Custom Dart Actions are the bridge between FlutterFlow's visual simplicity and Flutter's full coding power. Once you're comfortable creating, testing, and connecting them, you'll find that almost any feature — no matter how complex — becomes possible inside FlutterFlow.

Start small: pick one repetitive or complex piece of logic in your current project, convert it into a Custom Action following the steps above, and see the difference it makes in both performance and code cleanliness.

This tutorial is part of our internal training resource series. For questions or suggestions, reach out to the author.

Author: Wahab Waris